

Java Programming: Complete Study Guide

1. Introduction to Java

What is Java?

Java is a high-level, class-based, object-oriented programming language developed by **James Gosling** and his team at **Sun Microsystems** in **1991**. It was officially released to the public in **1995**. The language was originally called **Oak** (named after an oak tree outside Gosling's office) and was later renamed **Java** (inspired by Java coffee). Sun Microsystems was later acquired by **Oracle Corporation** in 2010, which now maintains and develops Java.

Java was designed with a central philosophy captured in its famous motto: "**Write Once, Run Anywhere**" (**WORA**). This means that Java code written on one platform (say, Windows) can run on any other platform (Linux, macOS, Android) without modification or recompilation, as long as a Java Virtual Machine (JVM) is available on that platform. This platform independence was revolutionary at the time and remains one of Java's most important characteristics.

Java draws its syntax heavily from **C and C++**, making it familiar to programmers with those backgrounds, but eliminates many of C/C++'s complex and error-prone features — most notably **manual memory management and pointers**. Instead, Java manages memory automatically through a built-in **Garbage Collector** that periodically identifies and frees memory that is no longer in use by the program.

Java is used across an enormous range of applications — from Android mobile apps to large-scale enterprise server software, from web applications to scientific computing. It consistently ranks among the top three most widely used programming languages in the world.

Java Features

Java's widespread adoption and longevity are due to a carefully designed set of features that make it robust, secure, portable, and productive.

Simple: Java was designed to be easy to learn, especially for programmers familiar with C or C++. It removes confusing and dangerous features of C++ such as operator overloading, multiple inheritance through classes, explicit pointers, and manual memory deallocation. The syntax is clean and consistent. The extensive standard library (Java API) provides ready-made solutions for common programming tasks so developers don't have to build everything from scratch.

Object-Oriented: Java is a fully object-oriented language — everything in Java is organized around objects and classes. The four fundamental principles of OOP — **Encapsulation** (bundling data and methods together, hiding implementation details), **Inheritance** (a class acquiring properties and behaviors of another class), **Polymorphism** (one interface, multiple implementations), and **Abstraction** (hiding complexity and showing only essential features) — are all core to Java programming. Even the basic entry point of a Java program (`main()`) must be inside a class.

Platform Independent: This is Java's defining characteristic. Java source code (.java files) is compiled not into machine-specific native code, but into **bytecode** (.class files). Bytecode is an intermediate, platform-neutral representation that can be executed by the **Java Virtual Machine (JVM)** on any operating system. Since JVMs exist for Windows, Linux, macOS, Solaris, and many other platforms,

the same bytecode runs everywhere. The JVM acts as an abstraction layer between the program and the hardware.

Secure: Java was designed with security as a priority from day one. There are no direct pointers to memory addresses — programs cannot access arbitrary memory locations, preventing buffer overflow attacks. The JVM provides a **Security Manager** that controls what a program is allowed to do. The **bytecode verifier** checks bytecode before execution to ensure it follows language rules and doesn't attempt illegal operations. Java's class loader mechanism provides controlled loading of classes.

Robust: Java emphasizes reliability and early error detection. The strong type system catches many errors at compile time rather than runtime. Java's **exception handling mechanism** (try-catch-finally) provides a structured way to handle runtime errors gracefully. Automatic garbage collection prevents memory leaks and dangling pointer errors that plague C/C++ programs. The elimination of pointers removes a whole category of hard-to-debug errors.

Architecture Neutral: Related to platform independence — the size of primitive data types in Java is fixed regardless of the platform. In C, the size of an int can vary between 16-bit and 32-bit systems. In Java, an int is always exactly 32 bits on every platform. This ensures consistent behavior across architectures.

Portable: The combination of platform independence and architecture neutrality makes Java programs truly portable. The same compiled bytecode file runs on any platform with a JVM. Java's standard library provides platform-independent APIs for I/O, networking, graphics, and more, so platform-specific code is rarely needed.

High Performance: While early Java was criticized for being slower than native C/C++ code (because of the overhead of interpretation by the JVM), modern JVMs use **Just-In-Time (JIT) compilation** which compiles frequently executed bytecode into native machine code at runtime. This dramatically improves performance, bringing Java close to native speeds for many workloads.

Multithreaded: Java has built-in, language-level support for **multithreading** — the ability to execute multiple parts of a program concurrently. The Thread class and Runnable interface are part of the core language. Java provides synchronization mechanisms (synchronized keyword, locks, atomic variables) to safely coordinate access to shared data between threads. This is essential for responsive user interfaces, server applications handling multiple simultaneous requests, and performance-critical applications.

Dynamic: Java supports dynamic loading of classes at runtime — a class can be loaded only when it is actually needed, rather than requiring everything to be loaded upfront. Java programs can dynamically discover and use new classes. This makes Java applications flexible and extensible — plugins, frameworks, and application servers rely on this capability.

Distributed: Java was designed for networked environments. The standard library includes extensive packages for networking (TCP/IP sockets, UDP), remote method invocation (RMI), and web protocols. Building distributed applications that communicate over networks is straightforward in Java.

Interpreted: Though Java uses JIT compilation for performance, the underlying execution model is still fundamentally interpreted — bytecode is interpreted by the JVM. This contributes to platform independence and makes certain dynamic features possible.

Java Applications

Java's versatility makes it suitable for a remarkably wide range of application types:

- **Desktop Applications** — Standalone GUI applications built using Java's built-in GUI toolkits: **AWT (Abstract Window Toolkit)** (older, maps to native OS widgets), **Swing** (platform-independent, more feature-rich), and **JavaFX** (modern, CSS-styled, multimedia-capable). Examples: development tools (NetBeans, Eclipse are themselves written in Java), office applications
- **Web Applications** — Server-side web development using **Servlets**, **JSP (JavaServer Pages)**, and frameworks like **Spring MVC**, **Struts**, and **Jakarta EE**. Java powers a huge portion of enterprise web backends worldwide
- **Enterprise Applications** — Large-scale, multi-tier business applications using **Java EE / Jakarta EE** platform — transaction management, distributed computing, EJBs, JPA for database access. Banking systems, insurance platforms, ERP systems commonly use Java
- **Mobile Applications** — **Android** mobile app development uses Java (and now Kotlin, which runs on the same JVM). Billions of Android devices run Java-based applications
- **Scientific Applications** — Java's robustness, portability, and mathematical libraries make it suitable for scientific computing, simulations, and research tools
- **Embedded Systems** — Java ME (Micro Edition) targeted resource-constrained devices. Java runs in smart cards, set-top boxes, and various embedded systems
- **Big Data Technologies** — Major big data processing frameworks are written in Java: **Apache Hadoop**, **Apache Kafka**, **Apache Spark** (partially)
- **Cloud and Microservices** — Frameworks like **Spring Boot** and **Micronaut** make Java a dominant language for cloud-native microservices development

2. Writing and Running the First Program

JDK, JRE, JVM

Understanding the relationship between JDK, JRE, and JVM is fundamental to understanding how Java works.

JVM — Java Virtual Machine:

The **JVM** is the cornerstone of Java's platform independence. It is an abstract computing machine — a software layer that simulates a virtual computer. The JVM's job is to execute Java bytecode. When you run a Java program, the JVM reads the .class bytecode file and executes the instructions.

The JVM is platform-specific — there is a different JVM implementation for Windows, Linux, macOS, etc. However, the bytecode it executes is platform-neutral. This is the magic of Java: the bytecode is the same everywhere, and the platform-specific JVM handles the translation to actual hardware instructions.

The JVM performs several important functions:

- **Class loading** — Loads .class files into memory using the ClassLoader subsystem

- **Bytecode verification** — Verifies that loaded bytecode is valid and safe before execution
- **Execution** — Interprets bytecode or compiles it to native code using the JIT compiler
- **Memory management** — Manages the heap (where objects are allocated), stack (where method call frames are stored), and other memory areas
- **Garbage collection** — Automatically identifies and recycles memory occupied by objects that are no longer reachable
- **Exception handling** — Manages the runtime exception mechanism

JRE — Java Runtime Environment:

The **JRE** is the complete runtime environment needed to **run** (execute) Java programs. It contains:

- The **JVM** (to execute bytecode)
- The **Java Class Libraries** — a large collection of pre-written Java classes organized into packages (java.lang, java.util, java.io, java.net, etc.) that programs can use
- Supporting files and configuration

If someone just wants to run a Java application (not develop one), they need the JRE. JRE = JVM + Class Libraries. It does not include development tools like the compiler.

JDK — Java Development Kit:

The **JDK** is the complete development package for Java developers. It contains everything needed to **develop, compile, debug, and run** Java programs:

- The **JRE** (which includes the JVM and class libraries) — so you can run programs
- **javac** — The Java compiler that compiles .java source files to .class bytecode files
- **java** — The launcher that starts the JVM and runs a Java program
- **javadoc** — Generates HTML API documentation from Java source comments
- **jar** — Creates and manages JAR (Java Archive) files
- **jdb** — The Java debugger
- Other development and diagnostic tools

Relationship: $\text{JDK} \supset \text{JRE} \supset \text{JVM}$ (JDK contains JRE, JRE contains JVM)

For development, you always install the JDK. For deployment (running apps on production servers), the JRE is sufficient.

Byte Code

Bytecode is the intermediate representation that Java source code is compiled into. It is neither human-readable source code nor machine-specific native binary code — it sits between the two. Bytecode files have the **.class extension**.

When you run `javac HelloWorld.java`, the compiler reads the Java source code, checks for syntax errors, performs type checking, and if everything is valid, produces a `HelloWorld.class` file containing bytecode. This bytecode is a set of instructions for the JVM — not for any specific physical CPU.

Why bytecode is important:

- **Platform independence** — Since bytecode is not tied to any specific hardware architecture, the same `.class` file can be shipped to any machine. The JVM on that machine interprets or JIT-compiles the bytecode to the local native instructions
- **Security** — The JVM's bytecode verifier checks bytecode before execution, ensuring it doesn't violate memory safety or type safety rules
- **Optimization** — Modern JVMs use sophisticated JIT compilers that analyze bytecode at runtime and compile frequently-executed "hot" sections into optimized native code, achieving near-native performance

Bytecode vs Source Code vs Native Code:

Aspect	Source Code (.java)	Bytecode (.class)	Native Code
Human readable	Yes	No	No
Platform specific	No	No	Yes
Executed by	N/A (compiled)	JVM	CPU directly
Performance	N/A	Moderate (JIT helps)	Fastest

Structure of a Java Program

Every Java program follows a specific, mandatory structure. Understanding this structure is essential before writing any Java code.

Package Declaration (optional): Packages are namespaces that organize related classes together, similar to folders in a file system. If present, the package declaration must be the very first statement. Example: `package com.example.myapp;`

Import Statements (optional): Import statements tell the compiler which classes from other packages are used in this file, so you don't have to write the fully qualified class name every time. Example: `import java.util.Scanner;`. The `java.lang` package is automatically imported — classes like `String`, `Math`, `System` don't need explicit imports.

Class Declaration (mandatory): Every Java program consists of at least one class. The class is declared with the `class` keyword followed by the class name. The class name must match the filename exactly (case-sensitive). For a public class named `HelloWorld`, the file must be named `HelloWorld.java`. The class body is enclosed in curly braces `{ }`.

The `main()` Method (mandatory for standalone programs): The `main()` method is the **entry point** of every standalone Java application — it is where execution begins when you run the program. Its signature is exactly: `public static void main(String[] args)`

Each part of this signature has a specific, required meaning:

- **public** — The method must be accessible from anywhere, including from outside the class (the JVM needs to call it)
- **static** — The method belongs to the class itself, not to any instance. The JVM calls `main()` without creating an object of the class first
- **void** — The method returns nothing
- **main** — The exact name the JVM looks for as the entry point
- **String[] args** — An array of String arguments passed from the command line when running the program. Can also be written as `String... args`

Method Body: The actual program code goes inside the `main()` method's body, enclosed in curly braces.

Hello World Program — Compile and Run

The traditional first program in any language is "Hello, World!" — a simple program that outputs the text "Hello, World!" to the screen.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Line by line explanation:

- `public class HelloWorld` — Declares a public class named `HelloWorld`. The file must be saved as `HelloWorld.java`
- `{` — Opens the class body
- `public static void main(String[] args)` — Declares the main method — the program's entry point
- `{` — Opens the main method body
- `System.out.println("Hello, World!");` — Prints the text "Hello, World!" followed by a newline to the standard output (console). `System` is a class in `java.lang`, `out` is a static field of type `PrintStream` representing standard output, `println()` is a method of `PrintStream` that prints and adds a newline
- `}` — Closes the main method body
- `}` — Closes the class body

Compiling the program: Open a terminal/command prompt, navigate to the directory containing `HelloWorld.java`, and run: `javac HelloWorld.java`

If there are no errors, the compiler creates HelloWorld.class in the same directory. If there are syntax errors, the compiler displays error messages with line numbers.

Running the program: After successful compilation, run: `java HelloWorld`

Note: You specify the class name (HelloWorld), not the file name (HelloWorld.class). The JVM automatically looks for the .class file. Output: Hello, World!

Common errors:

- Case sensitivity — helloworld.java won't match public class HelloWorld
 - Missing semicolon at end of statement — Java requires a semicolon after every statement
 - Running `java HelloWorld.class` instead of `java HelloWorld` — don't include the extension
-

3. Data Types and Variables

Primitive Types

Java has exactly **eight primitive data types** — the fundamental building blocks for storing simple values. They are called primitive because they are not objects — they directly hold their values in memory (on the stack, for local variables) and are not references to objects on the heap. Primitive types have fixed sizes across all platforms (a key feature of Java's portability).

byte:

- Size: 8 bits (1 byte)
 - Range: -128 to 127
 - Default value: 0
 - Used for: Saving memory in large arrays, processing raw binary data (files, network streams), when values are known to be small
 - Example: `byte age = 25;`
-

short:

- Size: 16 bits (2 bytes)
 - Range: -32,768 to 32,767
 - Default value: 0
 - Used for: Memory saving in large arrays where values fit within the range, legacy systems using 16-bit integers
 - Example: `short year = 2024;`
-

int:

- Size: 32 bits (4 bytes)
 - Range: -2,147,483,648 to 2,147,483,647 (approximately ± 2.1 billion)
 - Default value: 0
 - Used for: **The default and most commonly used integer type.** Counter variables, loop indices, array sizes, general integer arithmetic. Always use int when the value fits in the range
 - Example: `int population = 1000000;`
-

long:

- Size: 64 bits (8 bytes)
 - Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (approximately $\pm 9.2 \times 10^{18}$)
 - Default value: 0L
 - Used for: Values too large for int — timestamps in milliseconds, file sizes in bytes, large ID numbers, astronomical measurements. Long literals must be suffixed with L or l (uppercase L preferred to avoid confusion with the digit 1)
 - Example: `long population = 7800000000L;`
-

float:

- Size: 32 bits (4 bytes)
 - Precision: approximately 6-7 significant decimal digits
 - Default value: 0.0f
 - Used for: Saving memory when storing large arrays of floating-point numbers where single precision is sufficient. Float literals must be suffixed with F or f
 - Note: Not suitable for precise monetary calculations (use BigDecimal instead)
 - Example: `float temperature = 98.6f;`
-

double:

- Size: 64 bits (8 bytes)
- Precision: approximately 15-16 significant decimal digits
- Default value: 0.0d (the d suffix is optional)

- Used for: **The default and most commonly used floating-point type.** Scientific calculations, measurements, coordinates, any decimal values needing reasonable precision. More precise than float
 - Example: `double pi = 3.14159265358979;`
-

char:

- Size: 16 bits (2 bytes)
 - Range: 0 to 65,535 (unsigned)
 - Default value: `'\u0000'` (null character)
 - Used for: Storing a single character. Java uses **Unicode (UTF-16)** encoding, so char can represent characters from virtually any human language. Character literals are enclosed in **single quotes**
 - Example: `char grade = 'A';` or `char symbol = '\u0041';` (Unicode for 'A')
 - Note: char is technically an unsigned integer — arithmetic operations on chars are possible
-

boolean:

- Size: Not precisely defined by the JVM spec (typically 1 bit logically, but stored as 1 byte or 4 bytes in practice)
- Values: Only two possible values: true or false
- Default value: false
- Used for: Flags, condition variables, representing binary states (on/off, yes/no, valid/invalid)
- Example: `boolean isLoggedIn = true;`
- Note: Unlike C/C++, Java booleans cannot be converted to or from integers — you cannot use 1 for true or 0 for false

Summary table:

Type	Size	Range / Values	Default
byte	8 bits	-128 to 127	0
short	16 bits	-32,768 to 32,767	0
int	32 bits	~±2.1 billion	0
long	64 bits	~±9.2 × 10 ¹⁸	0L
float	32 bits	~6-7 decimal digits	0.0f
double	64 bits	~15-16 decimal digits	0.0d

Type	Size	Range / Values	Default
char	16 bits	0 to 65,535 (Unicode)	'\u0000'
boolean	~1 bit	true or false	false

Declaring and Initializing Variables

A **variable** is a named storage location in memory that holds a value. In Java, every variable must be **declared** before it can be used. Declaration tells the compiler the variable's name and data type, which determines how much memory to allocate and what operations are valid.

Declaration syntax: `dataType variableName;`

Initialization is assigning a value to a variable. Variables can be declared and initialized in the same statement or separately.

Key rules and concepts:

- Variables must be declared with a specific type and that type cannot change — Java is **statically typed**
- Local variables (inside methods) must be initialized before use. The compiler will refuse to compile code that uses an uninitialized local variable
- Instance variables (inside a class but outside methods) are automatically initialized to their default values (0 for numbers, false for boolean, '\u0000' for char, null for references) if not explicitly initialized
- Multiple variables of the same type can be declared in one statement, separated by commas:
`int x = 1, y = 2, z = 3;`
- Variable names are case-sensitive (age and Age are different variables)
- By convention, variable names use **camelCase** (start with lowercase, subsequent words capitalized): `studentName`, `totalMarks`, `isActive`
- Variable names cannot be Java keywords (`int`, `class`, `if`, `while`, etc.)
- Variable names must start with a letter, underscore (`_`), or dollar sign (`$`) — not a digit

Types of variables by location:

- **Local variables** — Declared inside a method or block. Exist only within that method/block. No default value — must be initialized before use
 - **Instance variables (fields)** — Declared inside a class but outside any method. Each object has its own copy. Have default values. Exist as long as the object exists
 - **Class variables (static fields)** — Declared with the `static` keyword inside a class. Shared by all instances of the class. Initialized when the class is loaded
-

The Scope and Lifetime of Variables

Scope refers to the region of the program where a variable is accessible — where it can be read or written. **Lifetime** refers to how long the variable exists in memory. In Java, scope is determined by the **block** (the pair of curly braces { }) in which the variable is declared.

Block scope: A variable declared inside a block { } is only accessible within that block and any nested inner blocks. Once the block ends (the closing }), the variable goes out of scope and cannot be accessed. This is the most fundamental scoping rule in Java.

Local variable scope: A local variable declared inside a method is accessible from the point of declaration to the end of the method. It does not exist before its declaration and cannot be accessed after the method returns.

Loop variable scope: A variable declared in the initialization part of a for loop (for (int i = 0; ...)) has scope limited to within the loop — the variable i does not exist before or after the for loop.

Nested block scope: A variable declared in an outer block is accessible in inner blocks (blocks nested within it). However, a variable declared in an inner block is not accessible in the outer block. You cannot declare a variable in an inner block with the same name as a variable in an enclosing block within the same method — the compiler flags it as a duplicate.

Instance variable scope: Instance variables (non-static fields) are accessible throughout the entire class in all non-static methods. Their lifetime is tied to the object — they exist as long as the object exists (is reachable).

Class variable scope: Static variables are accessible throughout the class and, if public, from outside the class using the class name. They exist for the entire lifetime of the program (from class loading to program termination).

Lifetime summary:

- Local variables: Created when declared, destroyed when the block ends
- Instance variables: Created when object is created (new), destroyed when the object is garbage collected
- Static variables: Created when the class is loaded, destroyed when the program ends

Type Conversion and Type Casting

Java is a **strongly typed language** — you generally cannot mix different types without explicit conversion. However, Java does allow type conversions under specific conditions.

Widening Conversion (Implicit / Automatic Type Conversion):

Widening conversion happens **automatically** when a value of a smaller type is assigned to a variable of a larger type. No data is lost, so the compiler performs this conversion silently without any special syntax. It is called "widening" because you are converting to a wider (larger) type.

The widening order is: byte → short → int → long → float → double (and char → int → long → float → double)

Examples:

- Assigning an int to a long — automatic, safe

- Assigning a float to a double — automatic, safe
- Assigning a byte to an int — automatic, safe
- In arithmetic expressions, if operands are of different types, the smaller is automatically widened to the larger type before the operation

Narrowing Conversion (Explicit Type Casting):

Narrowing conversion means converting from a larger type to a smaller type. This may result in **data loss** (loss of precision, truncation of higher bits, overflow) so Java requires you to explicitly acknowledge the conversion using a **cast**. The cast is written as the target type in parentheses before the value.

Syntax: `targetType variableName = (targetType) expression;`

Examples and effects:

- `(int) 3.99` → 3 (the fractional part .99 is simply truncated, not rounded)
- `(byte) 130` → -126 (130 exceeds byte range, the bits wrap around causing a completely different value)
- `(int) 'A'` → 65 (char to int gives the Unicode code point)
- `(char) 65` → 'A' (int to char gives the character for that code point)

Casting is your explicit acknowledgment that you understand potential data loss may occur. The compiler trusts you and performs the conversion.

String conversions:

- Primitive to String: Using `String.valueOf(value)`, `Integer.toString(value)`, or concatenating with an empty string `"" + value`
- String to primitive: Using wrapper class parse methods — `Integer.parseInt("42")`, `Double.parseDouble("3.14")`, `Boolean.parseBoolean("true")`

Integer arithmetic note: When you perform arithmetic with two int values, the result is int even if it would naturally be a larger type. For example, `int / int` gives int (integer division truncates the decimal). To get floating-point division, at least one operand must be a floating-point type: `(double) a / b` or `a / (double) b`.

Brief Overview of Operators

Operators are symbols that perform operations on values (called operands). Java has a rich set of operators organized into categories.

Arithmetic Operators:

Used for mathematical calculations on numeric values.

- `+` Addition — adds two values. Also used for String concatenation

- - Subtraction — subtracts right operand from left
 - * Multiplication — multiplies two values
 - / Division — divides left by right. For integers, performs integer division (truncates decimal). For floating-point, gives decimal result
 - % Modulus (remainder) — gives the remainder after integer division. $10 \% 3 = 1$. Useful for checking divisibility, cycling through ranges
 - + Unary plus — indicates positive value (rarely used explicitly)
 - - Unary minus — negates a value
-

Bitwise Operators:

Operate on individual bits of integer values. Used in low-level programming, flags, and optimization.

- & Bitwise AND — result bit is 1 only if both corresponding bits are 1
 - | Bitwise OR — result bit is 1 if either corresponding bit is 1
 - ^ Bitwise XOR (Exclusive OR) — result bit is 1 if bits are different
 - ~ Bitwise NOT (Complement) — inverts all bits
 - << Left shift — shifts bits left, filling with zeros on the right. Equivalent to multiplying by powers of 2
 - >> Right shift (signed) — shifts bits right, filling with the sign bit (0 for positive, 1 for negative)
 - >>> Unsigned right shift — shifts bits right, always filling with 0 regardless of sign
-

Relational (Comparison) Operators:

Compare two values and return a boolean result (true or false). Used in conditions.

- == Equal to — true if both operands are equal. For objects, compares references (memory addresses), not content
 - != Not equal to — true if operands are different
 - > Greater than
 - < Less than
 - >= Greater than or equal to
 - <= Less than or equal to
-

Logical Operators:

Combine boolean expressions to form compound conditions.

- **&&** Logical AND — true only if BOTH operands are true. **Short-circuit** — if left operand is false, right is not evaluated (since result is already false regardless)
- **||** Logical OR — true if AT LEAST ONE operand is true. **Short-circuit** — if left is true, right is not evaluated
- **!** Logical NOT — inverts a boolean. **!true** is false, **!false** is true

The short-circuit behavior of **&&** and **||** is important and useful — it prevents `NullPointerExceptions` in expressions like `object != null && object.getValue() > 0` (if `object` is null, the second part is never evaluated).

Assignment Operators:

Assign values to variables. All compound assignment operators combine an operation with assignment.

- **=** Simple assignment — assigns right value to left variable
- **+=** Add and assign — `a += 5` is equivalent to `a = a + 5`
- **-=** Subtract and assign — `a -= 3` is equivalent to `a = a - 3`
- ***=** Multiply and assign
- **/=** Divide and assign
- **%=** Modulus and assign
- **&=** Bitwise AND and assign
- **|=** Bitwise OR and assign
- **^=** Bitwise XOR and assign
- **<<=** Left shift and assign
- **>>=** Right shift and assign

Conditional (Ternary) Operator:

Java's only operator with **three operands**. It is a compact form of an if-else expression.

Syntax: `condition ? valueIfTrue : valueIfFalse`

The condition is evaluated. If it is true, the entire expression evaluates to `valueIfTrue`. If false, it evaluates to `valueIfFalse`. The result can be assigned to a variable or used directly in an expression.

Example: `int max = (a > b) ? a : b;` — assigns the larger of `a` and `b` to `max`.

It is called the ternary operator because it takes three operands.

Increment and Decrement Operators:

Shorthand operators that increase or decrease a variable's value by exactly 1.

- ++ Increment — increases value by 1
- -- Decrement — decreases value by 1

Both operators have two forms:

Pre-increment/decrement — The operator comes before the variable: ++a, --a. The variable is incremented/decremented first, and then the new value is used in the expression. So if a = 5, then b = ++a results in a = 6 and b = 6.

Post-increment/decrement — The operator comes after the variable: a++, a--. The current value is used in the expression first, and then the variable is incremented/decremented. So if a = 5, then b = a++ results in b = 5 (old value used) and then a = 6.

When the increment/decrement is used as a standalone statement (a++;), pre and post forms have identical effect.

Operator Precedence: Operators are evaluated in a specific order. Highest to lowest precedence (simplified): Postfix (a++) → Prefix (++a), Unary (~, !, -, +) → Multiplicative (*, /, %) → Additive (+, -) → Shift (<<, >>) → Relational (<, >, <=, >=) → Equality (==, !=) → Bitwise AND (&) → Bitwise XOR (^) → Bitwise OR (|) → Logical AND (&&) → Logical OR (||) → Ternary (?:) → Assignment (=, +=, etc.)

When in doubt about precedence, use parentheses to make the intended order explicit and improve code readability.

4. Control Flow

Control flow statements determine the **order in which statements execute**. Without control flow, a program executes every line exactly once from top to bottom. Control flow allows branching (executing different code based on conditions) and looping (repeating code), which are the foundations of all non-trivial programs.

if-else (Simple, Nested, and Ladder)

The **if-else statement** is the fundamental decision-making construct. It evaluates a boolean condition and executes one block of code if the condition is true and optionally another block if it is false.

Simple if: The simplest form — execute a block only if the condition is true. If the condition is false, the block is skipped and execution continues after it. The condition must evaluate to a boolean — unlike C/C++, Java does not allow non-zero integers as true conditions.

if-else: Adds an else clause — one block executes if the condition is true, the other if false. Exactly one of the two blocks always executes. Useful for binary decisions (is the number positive or not? Did the login succeed or fail?).

Nested if-else: An if or if-else statement placed inside another if or else block. Used when multiple layered conditions need to be checked. The inner if only executes when the outer condition is satisfied. Nesting can go arbitrarily deep, but deep nesting quickly becomes hard to read — more

than 2-3 levels suggests refactoring is needed. The **dangling else problem** — an else always pairs with the nearest preceding unmatched if, regardless of indentation.

if-else if ladder (Ladder): A series of if-else if conditions evaluated in order from top to bottom. As soon as one condition is true, its block executes and all remaining conditions are skipped. The optional final else at the bottom acts as a catch-all that executes only if none of the conditions above were true. Used when you have multiple mutually exclusive conditions to test — grading systems, menu selections, range categorizations.

Key points:

- Curly braces { } are technically optional when the if/else body is a single statement, but always using braces is strongly recommended to avoid subtle bugs (the "goto fail" bug in Apple's SSL implementation is a famous real-world example of a missing brace causing a critical security vulnerability)
 - Conditions must be boolean expressions — using = (assignment) instead of == (comparison) is a common bug that Java's compiler will often catch and warn about
-

switch-case

The **switch statement** provides an efficient multi-way branch — it selects one of many possible paths based on the value of a single expression. It is a cleaner alternative to a long if-else if ladder when branching depends on the exact value of a single variable.

How switch works:

The switch expression is evaluated once. Its value is compared against each case value in order. When a match is found, execution starts at that case and continues until a break statement is encountered (or the switch block ends). If no case matches, the optional default case executes.

Key concepts:

- **break statement** — Critically important. Without break, execution "falls through" to the next case, continuing to execute code regardless of whether that case matched. This fall-through is sometimes intentional (when multiple cases should execute the same code) but is usually a bug if unintentional. Always include break unless fall-through is deliberately intended
- **default case** — Executes when no case value matches the switch expression. Equivalent to the final else in an if-else ladder. Optional but recommended as a catch-all for unexpected values
- **Case values** — Must be compile-time constants — literal values or final variables. Cannot be variables, method calls, or ranges
- **Supported types** — byte, short, int, char, String (from Java 7), and enum types. Cannot use long, float, double, or boolean

Fall-through example use: Multiple cases can share the same code by stacking them without break between them — case 1: case 2: case 3: // shared code break; — all three values execute the same block.

switch vs if-else if:

- switch is generally faster for many cases (can be implemented as a jump table by the compiler)
- switch can only test equality against constants — if-else if can test any boolean expression including ranges, method calls, and compound conditions
- switch is cleaner and more readable when testing many exact values of a single variable

Enhanced switch (Java 14+): Modern Java introduced enhanced switch expressions using arrow syntax (case value -> action;) that eliminates fall-through entirely and can return a value, making switch both safer and more concise.

for loop

The **for loop** is used to execute a block of code a known number of times, or to iterate over a range of values. It is the most compact loop construct, combining initialization, condition, and update in one line.

Structure: for (initialization; condition; update) { body }

- **Initialization** — Executed once before the loop starts. Typically declares and initializes the loop counter variable. The variable declared here is scoped to the loop
- **Condition** — A boolean expression evaluated before each iteration. If true, the body executes. If false, the loop ends. If the condition is false from the start, the body never executes (zero iterations)
- **Update** — Executed after each iteration of the body. Typically increments or decrements the counter. Then the condition is re-evaluated
- **Body** — The statements executed in each iteration

Execution order: Initialization → Check condition → If true: execute body → execute update → check condition again → repeat. If false: exit loop.

Key characteristics:

- All three parts (initialization, condition, update) are optional. An empty condition is treated as always true (infinite loop: for(;;) { })
- Multiple variables can be initialized and multiple updates performed using the comma operator: for (int i = 0, j = 10; i < j; i++, j--)
- The loop variable is typically named i, j, k for simple counters by convention
- Preferred when the number of iterations is known before the loop starts

Enhanced for loop (for-each loop, Java 5+): for (dataType element : collection) { body } Iterates over every element in an array or collection without needing an index. Cleaner and less error-prone for simple traversal. Cannot be used when you need the index, when you need to modify the array, or when you need to iterate in reverse.

while loop

The **while loop** repeatedly executes its body as long as a condition remains true. Unlike the for loop, it has no built-in initialization or update — you manage these yourself. It is preferred when the number of iterations is not known in advance.

Structure:

```
while (condition) {  
    body  
}
```

- The condition is evaluated **before** each iteration (entry-controlled loop)
- If the condition is false from the very beginning, the body never executes — zero iterations are possible
- The body must eventually change something that makes the condition false, otherwise you have an infinite loop
- Initialization of any loop variable should be done before the while loop
- Update (incrementing the counter, reading new input, etc.) should happen inside the body

Common uses:

- Reading input until a sentinel value is entered (keep reading numbers until user enters -1)
- Waiting for a condition to become true (retry until a network connection is established)
- Processing items from a data structure until it is empty
- Game loops (keep playing until game over condition)

do-while loop

The **do-while loop** is a variation of the while loop with one critical difference: the body executes **first**, and then the condition is checked. This guarantees that the body executes **at least once** regardless of the condition.

Structure:

```
do {  
    body  
} while (condition);
```

Note the semicolon after the closing parenthesis — it is mandatory and a common mistake to omit it.

- The body executes, then the condition is evaluated
- If the condition is true, the body executes again. If false, the loop ends
- Even if the condition is false from the start, the body executes once before the condition is first checked

When to use do-while:

- Menu-driven programs — show the menu at least once, then ask if the user wants to continue
- Input validation — ask for input at least once, then repeat if invalid (do { get input } while (input is invalid))
- Any scenario where you know the action must be performed at least once before checking whether to repeat

Comparison of loop types:

Loop	Condition check	Guaranteed executions	Best for
for	Before (entry)	0 or more	Known number of iterations
while	Before (entry)	0 or more	Unknown iterations, may be zero
do-while	After (exit)	1 or more	Must execute at least once

Jump Statements: break and continue

break:

The break statement **immediately terminates** the innermost enclosing loop (for, while, do-while) or switch statement. Execution jumps to the first statement after the terminated loop/switch.

Uses of break:

- In switch statements: prevents fall-through to subsequent cases
- In loops: exit early when a condition is met (found what you're searching for, error occurred, user input to stop)
- Example: Searching through an array — once the target is found, break out of the loop rather than continuing to iterate unnecessarily

Labeled break: For nested loops, a regular break only exits the innermost loop. Java supports labeled break to exit a specific outer loop. A label is an identifier followed by a colon placed before the loop. break `labelName`; exits the labeled loop entirely. Labels are rarely needed in well-structured code but are useful when breaking out of deeply nested loops.

continue:

The continue statement **skips the rest of the current iteration** and moves to the next iteration of the loop. It does not terminate the loop — it just jumps to the next cycle.

- In a **for loop**: continue jumps to the update expression (i++) and then evaluates the condition
- In a **while/do-while loop**: continue jumps directly to the condition check

Uses of continue:

- Skip processing for items that don't meet a criteria (skip negative numbers when computing average, skip blank lines when reading a file)
- Avoid deeply nested if statements within loops — instead of `if (condition) { lots of code }`, use `if (!condition) { continue; }` and then write the code without nesting

Labeled continue: Like labeled break, `continue labelName;` skips to the next iteration of a specific outer labeled loop. Even more rarely used than labeled break.

Key difference: break exits the loop entirely, continue skips to the next iteration of the loop.

5. Arrays

1D Arrays: Declaration, Initialization, and Traversal

An **array** in Java is a **fixed-size, ordered collection of elements of the same data type**. Each element is stored at a specific position called an **index**, starting from **0** for the first element. Arrays provide a way to store and work with multiple values of the same type under a single variable name.

Key characteristics of Java arrays:

- Arrays are **objects** in Java — they are created on the heap with the `new` keyword
- Array **size is fixed** at creation time and cannot be changed later. If you need a resizable collection, use `ArrayList`
- All elements must be of the **same data type**
- Arrays are **zero-indexed** — the first element is at index 0, the last is at index `length - 1`
- Accessing an index outside 0 to `length-1` throws an **`ArrayIndexOutOfBoundsException`** at runtime
- The length of an array is accessible through the `length` property (not a method — no parentheses): `array.length`

Declaration:

You declare an array variable by specifying the data type followed by square brackets, then the variable name. The square brackets can go after the type or after the variable name (after the type is preferred for clarity).

`int[] numbers;` — declares a variable that can hold a reference to an int array. At this point, no array exists yet and the variable is null.

Creation (Instantiation):

An array is created using the `new` keyword, specifying the type and size. The size is the number of elements.

`numbers = new int[5];` — creates an array of 5 integers on the heap. All elements are automatically initialized to the default value for the type: 0 for int/byte/short/long, 0.0 for float/double, false for boolean, '\u0000' for char, null for object references.

Declaration and creation can be combined: `int[] numbers = new int[5];`

Initialization with values:

You can declare, create, and initialize an array with specific values in one step using an **array initializer** — a comma-separated list of values enclosed in curly braces. The size is automatically determined from the number of values:

`int[] numbers = {10, 20, 30, 40, 50};` — creates an array of 5 integers with the specified values.

This syntax is only valid at the point of declaration. To reinitialize later, you must use `new`: `numbers = new int[]{10, 20, 30, 40, 50};`

Accessing elements:

Individual elements are accessed using the array name followed by the index in square brackets: `numbers[0]` is 10, `numbers[4]` is 50. Elements can be read and written: `numbers[2] = 99;` changes the third element to 99.

Traversal:

Traversal means visiting every element of the array, typically to read or process each one.

Using a traditional for loop: Iterate from index 0 to `array.length - 1`. Use `array.length` for the bound to avoid hardcoding the size. The index variable gives you the position, which is useful when you need to know the index or when you want to modify elements.

Using the enhanced for loop (for-each): `for (int num : numbers) { // use num }` — cleaner syntax for simple traversal when you don't need the index. The loop variable `num` is a copy of each element — modifying `num` does not modify the array element.

Common array operations:

- Finding the largest/smallest element — traverse and track the maximum/minimum seen so far
- Computing sum and average — traverse and accumulate, then divide
- Searching (linear search) — traverse and compare each element with the target; break when found
- Copying an array — use `System.arraycopy()`, `Arrays.copyOf()`, or a loop (simple assignment `array2 = array1` does NOT copy — it just makes both variables point to the same array)
- Sorting — `Arrays.sort(array)` from `java.util.Arrays` sorts in ascending order
- Printing — `Arrays.toString(array)` from `java.util.Arrays` returns a readable string like `"[10, 20, 30]"`

2D Arrays: Introduction

A **2D array** (two-dimensional array) is essentially an **array of arrays** — it stores data in a tabular format with rows and columns, like a matrix or a grid. It is the natural data structure for representing tables, matrices, game boards (chess, tic-tac-toe), spreadsheets, and pixel grids.

Declaration:

`dataType[][] arrayName;` — The double square brackets indicate a 2D array. Each `[]` represents one dimension.

Creation:

```
arrayName = new dataType[rows][columns];
```

For example, `int[][] matrix = new int[3][4];` creates a 3-row, 4-column grid of integers, for a total of 12 elements. All elements initialize to 0.

Initialization with values:

A 2D array initializer uses nested curly braces — each inner set represents one row:

```
int[][] grid = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

This creates a 3×3 grid. `grid[0]` is the first row `{1, 2, 3}`, `grid[1][2]` is the element at row 1, column 2, which is 6.

Accessing elements:

`arrayName[rowIndex][columnIndex]` — Both indices are zero-based. For a 3×4 array, valid row indices are 0-2 and valid column indices are 0-3.

Dimensions:

- `matrix.length` gives the number of rows
- `matrix[i].length` gives the number of columns in row `i` (in a regular rectangular array, this is the same for all rows)

Traversal:

2D arrays are typically traversed using **nested for loops** — an outer loop iterating over rows and an inner loop iterating over columns within each row.

```
for (int i = 0; i < matrix.length; i++) {    // rows  
    for (int j = 0; j < matrix[i].length; j++) { // columns  
        // access matrix[i][j]  
    }  
}
```

The enhanced for loop can also traverse a 2D array: `for (int[] row : matrix) { for (int element : row) { }`

Jagged arrays:

In Java, 2D arrays are technically arrays of arrays, which means each row can have a **different length** — these are called jagged arrays. You create them by specifying only the number of rows in the outer array, then creating each row separately with its own size:

```
int[][] jagged = new int[3][];
```

```
jagged[0] = new int[2];
```

```
jagged[1] = new int[4];
```

```
jagged[2] = new int[1];
```

This flexibility (arrays of different row lengths) is unique to Java's implementation of multi-dimensional arrays compared to some other languages.

Common 2D array uses:

- Matrix arithmetic (addition, multiplication)
- Representing game boards
- Storing tabular data (rows of records)
- Image processing (2D pixel grids)
- Dynamic programming problems (memoization tables)

Memory layout: A Java 2D array `int[3][4]` is not stored as a contiguous block of 12 integers in memory (as in C/C++ 2D arrays). It is stored as: one array of 3 references (the outer array), pointing to three separate arrays of 4 integers each (the inner arrays). This is what enables jagged arrays but means memory access patterns are slightly different from C-style 2D arrays.